

The background of the entire image is a dark blue field filled with a pattern of red dots. These dots are arranged in a way that they form a large, faint, stylized circular shape in the center, with the density of the dots increasing towards the center.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

REAL-TIME NEWS TREND & SENTIMENT ANALYSIS PLATFORM

Big Data

Group 12

Nguyen Mau Trung – 20225534

Luong Huu Thanh – 20225458

Doan Anh Vu – 20225465

Vu Trung Thanh – 20220066

Nguyen Tien Thang – 20225530

ONE LOVE. ONE FUTURE.

1. Problem Definition
2. Architecture and Design
3. Implementation Details
4. Lessons Learned
5. Dashboard Demonstration

Modern Digital Media Challenges:

- ▶ Millions of articles published daily across thousands of sources
- ▶ Traditional methods (manual curation, keyword alerts) cannot keep pace
- ▶ 24/7 news cycle requires real-time situational awareness

Three Core Problems Identified:

1. Real-time sentiment monitoring lacks scalability
2. Category classification is labor-intensive and inconsistent
3. Historical and real-time analytics exist in silos

Problem 1: Sentiment Monitoring

- ▶ Manual sentiment assessment is slow and cannot scale
- ▶ Organizations miss early warning signals (e.g., sentiment shift from $+0.4$ to -0.2)

Problem 2: Category Classification

- ▶ Manual categorization is error-prone across diverse topics
- ▶ Need to detect 300% volume increases in specific categories

Problem 3: Unified Analytics

- ▶ Separate systems for archival and real-time monitoring
- ▶ **Goal:** Merge real-time data (72h) with historical baselines (months)

Why Big Data Technologies?

- ▶ High-velocity news streams (1000s articles/hour)
- ▶ Large-scale historical archives
- ▶ Complex analytical workflows

Key Technologies:

- ▶ Apache Kafka (message buffer)
- ▶ Spark Structured Streaming
- ▶ Spark Batch Processing
- ▶ MongoDB (serving layer)

Lambda Architecture

Combines batch and stream processing:

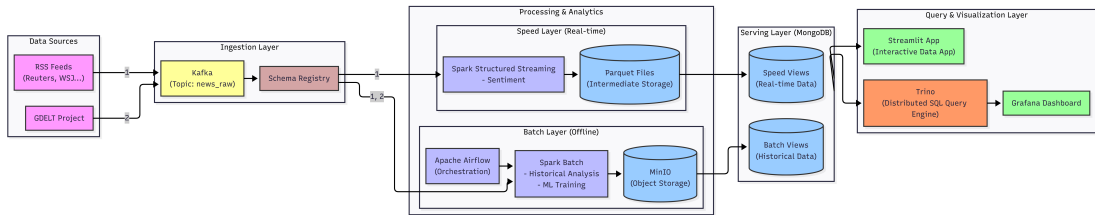
- ▶ **Batch Layer:** Accuracy
- ▶ **Speed Layer:** Freshness
- ▶ **Serving Layer:** Unified queries

Four Logical Layers:

1. **Ingestion Layer:** RSS feeds, GDELT API → Kafka
2. **Speed Layer:** Real-time processing via Spark Streaming
3. **Batch Layer:** Historical processing via Spark Batch
4. **Serving Layer:** Streamlit + Grafana dashboards

Data flows from sources through processing layers to unified visualization.

Overall System Architecture



Lambda Architecture Overview

Ingestion Layer Components:

- ▶ **News Crawlers:** Continuous polling of RSS feeds (Reuters, WSJ, BBC)
- ▶ **Apache Kafka:** Central message broker with 48-hour retention
- ▶ **Schema Registry:** Avro schema management for compatibility

Data Routing Strategy

- ▶ **RSS Feeds** → Both Speed Layer (real-time) and Batch Layer (historical)
- ▶ **GDELT API** → Batch Layer only (historical event data)

Real-Time RSS Processing Pipeline:

Kafka $\xrightarrow{\text{Consume}}$ Spark Streaming $\xrightarrow{\text{Write}}$ Parquet $\xrightarrow{\text{Load}}$ MongoDB

Processing Functions:

- ▶ Keyword-based sentiment scoring
- ▶ Lightweight category classification
- ▶ Keyword extraction (top-5)

Output to MongoDB:

- ▶ news_rt database
- ▶ 3-day TTL lifecycle
- ▶ Upsert pattern for idempotency

Historical Data Processing Pipeline:

Kafka (RSS + GDELT) $\xrightarrow{\text{Airflow}}$ Spark Batch $\xrightarrow{\text{Write}}$ MinIO $\xrightarrow{\text{Export}}$ MongoDB

Airflow DAG Tasks (Daily at 6 PM):

1. `crawl_fresh_news`: Fetch latest RSS + GDELT articles
2. `process_sentiment`: Execute batch NLP processing
3. `classify_articles`: ML-based category prediction
4. `integrated_analytics`: Graph + Time-series analysis
5. `cleanup_rt_data`: Remove data older than 3 days

Sentiment Analysis (Speed Layer):

$$S_{speed} = \frac{|W_{positive} \cap T| - |W_{negative} \cap T|}{|W_{positive} \cap T| + |W_{negative} \cap T| + 1} \quad (1)$$

Category Classification:

- ▶ **Speed Layer:** $C = \arg \max_c |K_c \cap T|$ (keyword matching)
- ▶ **Batch Layer:** $C = \arg \max_c P(c | \text{TF-IDF}(T))$ (Logistic Regression)

Trend Detection - Burstiness Coefficient:

$$B = \frac{\sigma}{\mu} \quad (\text{High } B \text{ indicates volatile, trending topics}) \quad (2)$$

RSS Feed Crawler:

- ▶ Mode: STREAM (60s polling) or BATCH (single-run)
- ▶ Unique article ID generation
- ▶ Content hash for deduplication
- ▶ Exponential backoff retry (1s-30s)

GDELT Crawler:

- ▶ Historical event data retrieval
- ▶ Batch-only mode for Airflow
- ▶ Tagged for Batch Layer consumption

Schema Registry:

- ▶ Avro schema evolution
- ▶ Backward compatibility

Sentiment Analysis UDF:

Code Example

```
POSITIVE_WORDS = {'surge', 'gain', 'rise', 'growth'...}  
NEGATIVE_WORDS = {'drop', 'fall', 'decline', 'crash'...}
```

```
def analyze_sentiment(text):  
    pos = sum(1 for w in tokens if w in POSITIVE)  
    neg = sum(1 for w in tokens if w in NEGATIVE)  
    return (pos - neg) / (pos + neg + 1)
```

Category Classification: Maps keywords to categories (e.g., “election” → “Politics”)

Consolidated Analytics Job (`advanced_aggregations.py`):

ML Classification

- ▶ Tokenizer
- ▶ TF-IDF Features
- ▶ LogisticRegression
- ▶ CrossValidator (3 folds)

Graph Analytics

- ▶ GraphFrames
- ▶ Source-entity graphs
- ▶ PageRank algorithm
- ▶ Co-occurrence analysis

Time-Series

- ▶ Rolling statistics
- ▶ Burstiness (σ/μ)
- ▶ Temporal trends
- ▶ Window functions

~30% code reduction through consolidation

Streamlit Dashboard:

- ▶ 30 visualization functions
- ▶ PyMongo for MongoDB queries
- ▶ Plotly for interactive charts
- ▶ Time range filters (1h to All)

Key Functions:

- ▶ `render_sentiment_distribution()`
- ▶ `render_category_chart()`
- ▶ `render_keywords_chart()`
- ▶ `render_realtime_trends()`

Grafana + Trino:

- ▶ Trino SQL engine over MongoDB
- ▶ Time-series charts
- ▶ Heatmaps and alerts
- ▶ Operational monitoring

Lambda Merge

Query-time merge:

- ▶ Speed Layer: last 3 days
- ▶ Batch Layer: older data

22 Kubernetes Manifests in k8s/ directory:

Infrastructure:

- ▶ Kafka + Zookeeper
- ▶ MongoDB StatefulSet
- ▶ Spark Master/Workers
- ▶ MinIO Object Storage
- ▶ Trino + Grafana
- ▶ Airflow Scheduler

Application Workloads:

- ▶ News Crawler Deployment
- ▶ Streamlit Dashboard
- ▶ Streaming Pipeline Job
- ▶ Batch Processing CronJob
- ▶ ML Training Job
- ▶ Alert Consumer

ConfigMaps: Code mounted as volumes (crawler-code, spark-jobs-code, avro-schemas)

Lesson 1: Multilingual Data Handling

Problem:

- ▶ Domain filtering insufficient
- ▶ International subdomains serve non-English content
- ▶ NLP models failed on foreign text

Approaches Tried:

- ▶ Strict domain allowlist → Failed
- ▶ langdetect library → +150ms latency

Solution

Lightweight filter using feed metadata:

```
if language.lower() != 'english':  
    return None
```

Result: 99.5% English content, no latency overhead

Takeaway: Filter at earliest point (ingestion) to save resources

Problem: Separate Spark jobs with duplicated data loading patterns

Challenge:

- ▶ Schema changes required 3+ file edits
- ▶ Technical debt accumulation
- ▶ Code divergence over time

Solution

Consolidated into
`advanced_aggregations.py`:

- ▶ Single orchestrator job
- ▶ Imports specialized modules
- ▶ ~30% LOC reduction
- ▶ DAG simplified: 8 → 5 tasks

Takeaway: Treat Spark jobs as software modules, not standalone scripts



Lesson 3: Real-Time Category Classification

Problem: Streaming pipeline lacked heavy ML model

- ▶ Real-time articles defaulted to “Uncategorized”
- ▶ Dashboard category charts were useless for real-time data

Approaches Tried:

- ▶ Full MLlib model in streaming → OOM kills, >2s latency
- ▶ Default all to “Uncategorized” → Unhelpful

Solution: Keyword-based UDF

Lightweight classification with <5ms latency per article
“election” → Politics, “stock” → Finance, etc.

Takeaway: Speed Layer requires approximation over perfection



Lesson 4: Serving Layer Optimization

Problem: Dashboard needed low-latency access to batch data

- ▶ Parquet reads via PyArrow: 5-10s latency, concurrency issues
- ▶ Query times exceeded 10 seconds

Solution: Dual-Write Strategy

Modified `batch_pipeline.py` to write to BOTH:

- ▶ **MinIO/Parquet:** Archive for data scientists
- ▶ **MongoDB:** Serving for dashboard queries

Result: Query latency reduced from 10s to $<1s$

Takeaway: Write for the reader - optimize storage based on access patterns



Lesson 5: Airflow DAG Synchronization

- ▶ Problem: Code changes not reflected in K8s pods
- ▶ Solution: ConfigMaps with `create-configmaps.sh` script

Lesson 6: Minikube Resource Exhaustion

- ▶ Problem: Pods in CrashLoopBackOff due to memory pressure
- ▶ Solution: Scale down non-essential services (freed ~4GB RAM)

Lesson 7: Silent Data Failures

- ▶ Problem: Dashboard showed stale data from 2023
- ▶ Solution: Date validation + prominent “Last Update” KPI

Lesson 8: Vertical vs. Horizontal Scaling

- ▶ Problem: Adding executors caused memory thrashing
- ▶ Solution: Local mode `local[2]` reduced job time from 20+ to <5 minutes
- ▶ *For small data (<10GB), single-node is often faster*

Lesson 9: UTF-8 Encoding

- ▶ Problem: Emojis displayed as “???” in dashboard
- ▶ Solution: `ENV LANG=C.UTF-8` in all Dockerfiles

Lesson 10: Automated Data Lifecycle

- ▶ Problem: Real-time collection grew unbounded
- ▶ Solution: Airflow cleanup task (3-day retention)
- ▶ *Data requires garbage collection like memory*

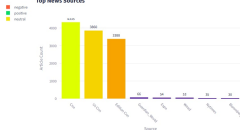


Dashboard Screenshots

Sentiment Distribution



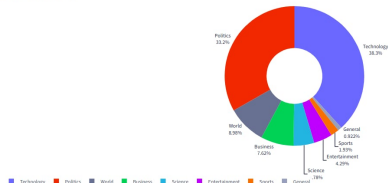
Top News Sources



News Volume Over Time



Category Distribution



Interactive Streamlit dashboard with real-time and historical data visualization

Member Name	Task Description
Nguyen Mau Trung	Infrastructure and Deployment (Kubernetes setup and Kafka broker management)
Luong Huu Thanh	Data Storage (Minlo/MongoDB implementation and schema definition)
Doan Anh Vu	Batch Layer (Spark batch processing and Airflow development)
Vu Trung Thanh	Speed Layer (Spark Structured Streaming and real-time aggregation)
Nguyen Tien Thang	Visualization (Triton integration and dashboard development)

A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in concentric, slightly offset rings, creating a sense of depth and movement. The word "HUST" is centered within this pattern.

HUST

THANK YOU !

